

QRSMS: A SMS client with end-to-end encryption via Quick Response (QR) code

John Mel R. Ramos and Jaderick P. Pabico

Abstract—Short message service (SMS) remains a popular option for text communication despite the existence of online messaging platforms. Whether because of its convenience, practicality, or accessibility, SMS continues to be a choice for different application. However, securing the SMS network remains a challenge because in its inception, security was not made a priority for its development. This paper introduced a software based approach to solving the lack of end-to-end security for SMS. QRSMS is an android application that enables users to create an end-to-end encrypted channel sent through SMS. End-to-end encryption is enabled through the exchange of QR codes with the contact that they want to establish a secure channel with. Key-agreement is achieved using the Elliptic Curve Diffie-Hellman (ECDH) while key pairs are generated using Elliptic Curve (EC). SMS messages are encrypted using the Advance Encryption Standard (AES) using a key derived from the key-agreement protocol. A usability study was conducted to gather an understanding about the system's usability and ease-of-use. On average, users generally find the system with relative ease to use, however, the mean usability score from the System Usability Scale was average at best. Recommendations for future researches and further improvements are discussed at the end of this paper.

Index Terms—SMS, end-to-end encryption, QR Code, Android

I. INTRODUCTION

Despite the emergence of instant messaging (IM) and Over-the-top (OTT) messaging applications, short message service (SMS) remains as a popular option for mobile communication. With the first text message sent in 1992 through the Global System for Mobile communication (GSM) network, the technology has seen a lot of growth and development [1]. While mostly used for person-to-person communication [2], the medium has seen other applications such as alert notification [3], [4], empowering businesses and online services [5]–[7], mobile health (m-health) interventions [8], [9], mobile government applications [10], [11], education [12], Two-Factor Authentication (2FA) [13] and more. During the year 2022, Globe Telecom gained Php 8.8 billion revenue from SMS with 86.7 million mobile customers. Furthermore, the company has invested Php 1.1 billion to improve spam and fraudulent SMS detection and prevention [14]. Unreliable internet service as well as the lack of Internet infrastructure in some area remains a challenge in the adoption of online services for communication such as Facebook Messenger, Viber, Telegram, etc. SMS on the other hand is ubiquitous.

Presented to the Faculty of the Institute of Computer Science, University of the Philippines Los Baños in partial fulfillment of the requirements for the Degree of Bachelor of Science in Computer Science

Anyone with a mobile device and a registered subscriber identity module (SIM) card can send and receive text messages regardless of their platform and cellular provider [15].

Regardless of its wide range of applications and sustained use, however, SMS is not ideal for private communication such as sending sensitive and confidential information as early specifications for SMS does not have security in mind [16]. The European Telecommunications Standards Institute (ETSI) specified the use of A5 algorithm for encryption over the GSM network together with the A3 and A8 algorithm for authentication and cipher key generation to provide security [17] but attacks on the A5/1 and the weaker A5/2 algorithms has been demonstrated even with minimal resources [18]–[20] and even the A5/3 used for Third Generation (3G) network has been compromised [21]. Mobile network operators may also opt not to use any encryption during the transit of message wirelessly [22]. This means that a Man-in-the-Middle (MITM) can sit between the Mobile Equipment (ME) and Base Station Subsystem (BSS) using an International Mobile Subscriber Identity (IMSI) catcher to actively intercept traffic and even request the victim ME to use a weaker or no encryption at all [23]. Additionally, encryption is only performed between the Mobile Equipment (ME) and the Base Station Subsystem (BSS) [17]. Once the message reaches the SMS Center (SMSC), messages are stored and kept in plain text for a while allowing malicious entities to target the SMSC to view the contents of these messages. This makes an end-to-end encryption a huge improvement over what is available in the GSM specifications.

To provide a better security system for SMS, independent researchers have proposed frameworks and protocols that offers end-to-end encryption. PK-SIM [24] provides end-to-end security between the mobile user and service provider through Public Key Infrastructure (PKI). SMSec [25] was developed using the Java Wireless Messaging API (WAP) and uses a combination of asymmetric and symmetric cryptography to provide security. EasySMS [26] and SecureSMS [7] are completely based on symmetric key cryptography. Similarly, SmartSMS [27] uses symmetric key encryption for m-health application. These protocols however, requires additional resources and infrastructure on the side of the service provider in order to be implemented.

This paper proposed an SMS client that uses quick-response (QR) codes for key exchange. Initially developed by Toyota's subsidiary Denso Wave for inventory of vehicle parts manufacturing, QR code has seen many applications in today's era [28]. Able to store up to a maximum of 7,089 characters

in one code [29] it can be used to encode text, URL, and other data and can be decoded using a mobile device equipped with a camera and the appropriate software which can display a text, connect to a wireless network, open a webpage, or open a linked application [30]. This client encodes the phone number public key needed to generate a shared secret in a QR code. The generated code can be shared to the intended recipient through offline (in person) or other secure online platform that allows sending of images.

End-to-end security is achieved through the combined use of symmetric and asymmetric cryptography as well a key-agreement protocol. Advanced Encryption Standard (AES) is used for encrypting the message as it provides the most security while being performant [31]. Elliptic Curve Cryptography (ECC) is used for generating key pairs and Elliptic Curve Diffie-Hellman (ECDH) for generating a shared secret. The use of ECC allows for a smaller key length while providing a comparable security as the Rivest–Shamir–Adleman (RSA) public key cryptosystem [32].

The proposed method used the android platform to develop an SMS client that can generate key pair, exchange public keys through QR code, and establish an end-to-end encrypted communication.

II. LITERATURE REVIEW

Regardless of the insecurity present in SMS, due to its widespread application, it remains to be of use to many users around the world. To provide better end-to-end security, protocols/frameworks has been proposed.

PK-SIM card is a framework developed based on Public Key Infrastructure (PKI) providing end-to-end encryption between the Service Provider (SP) and the mobile user which contains the PK-SIM card. Their framework is composed of the mobile client, a Certification Authority and Registration Authority (CA/RA), a Secure Access gateway (SAG) found in the service provider's infrastructure and the Mobile Operator (MO) that provides the network for SMS communication. Before the user can send secure messages, the SAG together, with the CA/RA must first validate the identity of the mobile client. A primary key shared between the client and the SAG is generated during the authentication phase. This key is used to establish a session key which is used to symmetrically encrypt/decrypt the message. However, in their approach, end-to-end security is only available between the mobile client and the SAG on the Service Provider's side. Additionally, it is stated that the generation of primary key in the authentication phase takes up to a day or longer. Every time this key expires, the protocol must be performed again from the beginning [24]. Similarly, the SMSec protocol is designed such that it uses a combination of asymmetric cryptography for authenticating the user and uses symmetric cryptography for the actual exchange of messages. It also falls short such that end-to-end encryption is between the mobile device and the authentication source. Unlike PK-SIM however, SMSec is a software solution built with the Java Wireless Messaging API (WAP) [25].

In contrast, the EasySMS protocol is designed to provide end-to-end encryption using only symmetric algorithm. In their

approach, an Authentication Server (AS) is responsible for the storage of all secret key shared between the AS and the respective Mobile Station (MS) or the user. It is also assumed that there is a shared secret key between the AS and a CA/RA which contains information about every mobile subscriber. Before a SIM card could be used, a subscriber must first register within the CA/RA. With these requirements met, the AS, with the help of CA/RA can now authenticate any MS that wants to send an SMS securely. However, this approach falls under the same disadvantage as the previous protocols. When user A wants to send a message to user B, the AS validates the identity of both users and generate different key for each respective user. This means that the message from user A is encrypted with the key shared between the AS and user A, decrypted in the AS then re-encrypt the message for user B using the key shared between the AS and user B [26]. This approach is also used for the development of SecureSMS but is proposed to be specific for value added services (VAS) and commercial applications [7]. SmartSMS follows a similar concept but instead of the AS being connected to a CA/RA, a trusted third party hosted in a public cloud is responsible for the functions of CA/RA [27].

In these protocols, apart from being not truly end-to-end encrypted (encryption only occurs between the mobile station and the service provider), they also require a third party for authenticating the user. This means that in order to achieve secure communication, changes within the service provider's and the network itself must be considered. This paper will introduce a different approach that does not require additional infrastructure by having the cipher text sent through the SMS network while encryption and decryption is performed by the communicating devices.

III. OBJECTIVES

To create a secure platform for the exchange of encrypted SMS, the main objective of this research is to develop an SMS client application that can encrypt and decrypt messages and utilize QR codes as means of exchanging keys for generating a shared secret. To be specific, this research aims to do the following:

1. Design and implement a key-agreement protocol using QR code as means for key exchange.
2. Design and implement an SMS client for mobile android device with end-to-end encryption.
3. Conduct a usability study of the SMS client application.

IV. MATERIALS AND METHODS

A. Key-Agreement Protocol

To achieve end-to-end security, participants in the exchange should be able to generate a shared key known only to them. This process can be further divided into two steps. Keypair generation and key-exchange. This two step process is shown in Figure 1.

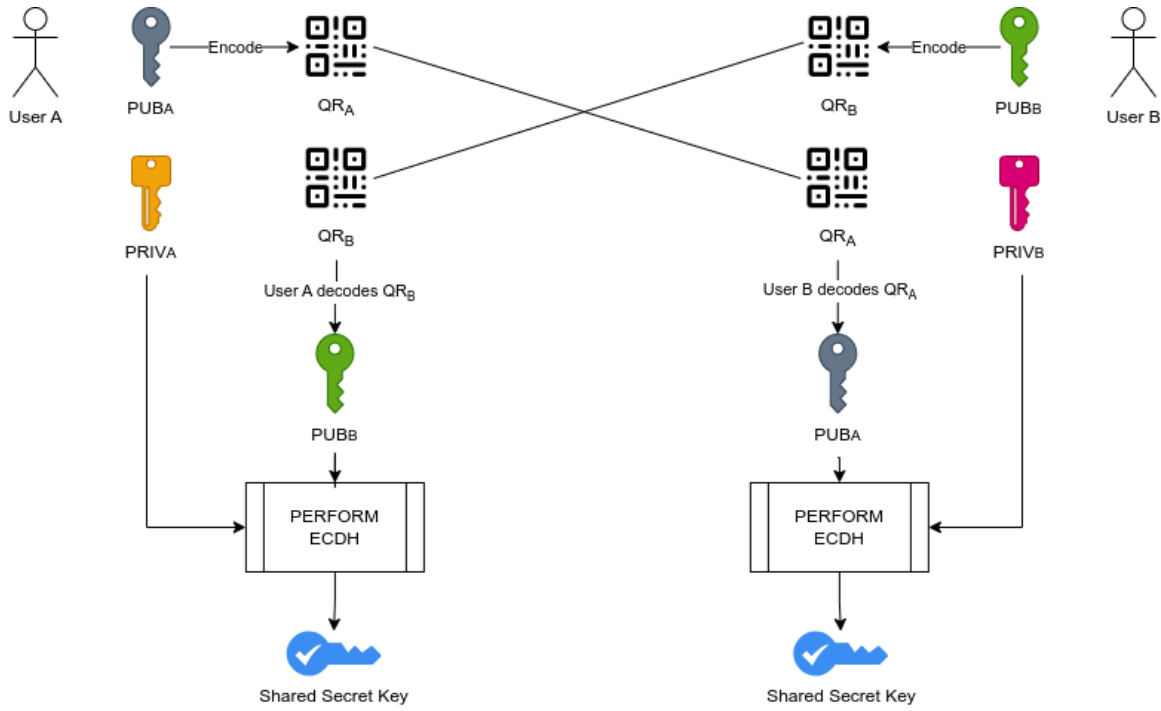


Fig. 1. Overview of key generation and key agreement in QRSMS

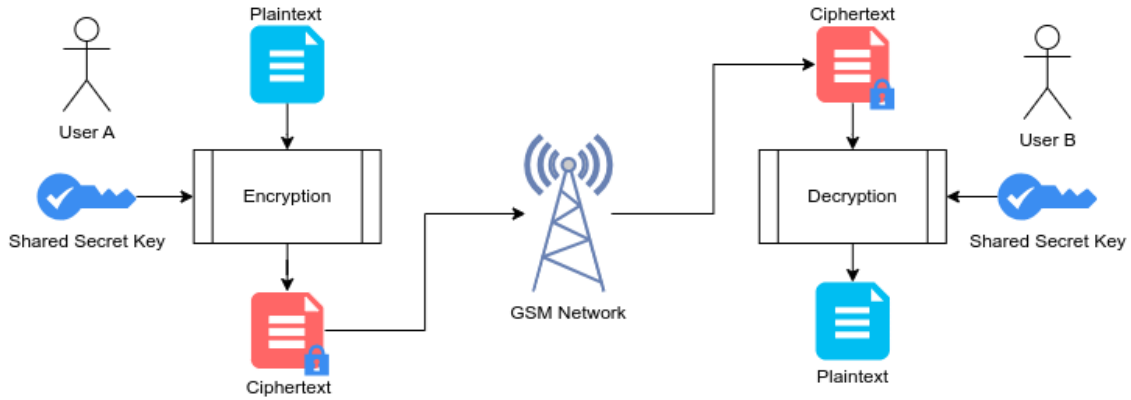


Fig. 2. Overview of sending and decryption of messages in QRSMS

1) *Key pair Generation:* To be able to generate a shared secret with another user, each party involved generates a public and private key pair for the other contact. Key pair generation uses the Elliptic Curve (EC) algorithm to create an EC key pair under the secp256r1 curve. While the key pair generation remains largely the same for any device supported by the application, the way it is stored is different depending on the android device's API level either in the Android Keystore system [33] or room database [34] encrypted using SQLCipher for android [35].

The generated public key, along with the user's phone number is then encoded into a QR code using the ZXing [36] library. The complementing private key is kept securely in the appropriate storage system to be used for finalizing the key exchange.

2) *Key Exchange:* With each party having their QR code for the other, they will exchange their QR codes through in-person or online as long as they can verify that the QR code belongs to the indicated contact. The given QR code can be scanned using the device's camera, decoded by Google's ML kit analyzer [37] with the barcode scanning [38] api. This QR code should contain the contact's phone number and public key string.

After a successful scan, the system looks for a corresponding private key associated with the contact. This allows the system to proceed with the key exchange using ECDH key agreement protocol and generate a shared secret key that should be the same for both users involved. This secret key is then fed into a HMAC-based Extract-and-Expand Key Derivation Function (HKDF) to create the final key that will

be used for AES encryption and decryption.

B. SMS Client

The SMS client is developed for the Android operating system with a minimum API level of 24 with the help of the Android Studio integrated development environment (IDE). Written in the Kotlin programming language, this allows full compatibility with Java libraries while being less verbose and fully supported for android development [39]. The user interface (UI) is written using the Jetpack Compose which is the recommended UI framework for new android application [40].

QRSMS application is built with features that allow users to send and read regular and encrypted SMS message, generate and scan QR codes and management of existing keys stored in the system. To enable end-to-end encryption, both the sending and receiving party must have the application installed and have exchanged their QR code generated by the application through means suggested above. In case only one party has the software installed, if they decide to send an encrypted message to a user without the application, or the recipient have not yet established the shared secret, the latter might receive the message in cipher text, unable to read said text. If the opposite were to occur, the application should be able to determine if the received text was encrypted or not.

When each of the user involved in the key-agreement have succeeded in creating a shared secret with their intended contact, the exchange of encrypted SMS is made available. AES algorithm with cipher block chaining (CBC) mode was used for symmetric cryptography using the derived secret key as the key for cryptographic operation shown in fig. 2.

C. Usability study for QRSMS

Throughout the duration of the development process, the features and interface of the system are thoroughly tested to ensure that the program can perform its intended functions. However, upon the completion of the minimum viable product, usability test was conducted to gain regular users' perspective with regard to their thoughts on how usable and easy to use they think the system is. Users was asked to perform various tasks that involves the following capabilities of the application:

1. Generation of QR code containing the public key to be used for a contact
2. Scanning and decoding of QR code received from a contact
3. Sending of unencrypted and encrypted messages
4. Reading of unencrypted and encrypted messages
5. Management of stored keys

Testers were invited through a social media post containing the details of the test, and links to the questionnaire. Anyone interested and willing can participate in testing the application granted that they meet the following requirements:

1. Users has an android device with version 7 (Nougat) or higher.
2. Users are registered to a postpaid plan or an unlimited text to all network package from any mobile provider.
3. Users device has a working rear camera.

System Usability Scale

© Digital Equipment Corporation, 1986.

	Strongly disagree				Strongly agree
1. I think that I would like to use this system frequently	☐	☐	☐	☐	☐
2. I found the system unnecessarily complex	☐	☐	☐	☐	☐
3. I thought the system was easy to use	☐	☐	☐	☐	☐
4. I think that I would need the support of a technical person to be able to use this system	☐	☐	☐	☐	☐
5. I found the various functions in this system were well integrated	☐	☐	☐	☐	☐
6. I thought there was too much inconsistency in this system	☐	☐	☐	☐	☐
7. I would imagine that most people would learn to use this system very quickly	☐	☐	☐	☐	☐
8. I found the system very cumbersome to use	☐	☐	☐	☐	☐
9. I felt very confident using the system	☐	☐	☐	☐	☐
10. I needed to learn a lot of things before I could get going with this system	☐	☐	☐	☐	☐

Fig. 3. Standard System Usability Scale [41]

The test was open for a week and qualified participants can do the test at anytime to their liking.

The questionnaire was given as a Google form and can be done by the user without the researcher's guide. The form also includes instructions that can help them get started in the unmoderated test. Single Ease Question (SEQ) was asked right after completing each task which aims to understand how easy or difficult they found each task to be. This is followed by a standard System Usability Scale (SUS) (fig. 3) after completing the tasks to assess the system's perceived usability.

V. RESULTS AND DISCUSSION

A. Key-agreement protocol with QR Codes

To allow users exchange end-to-end encrypted messages, each participant in a conversation must first generate an elliptic curve (EC) keypair for their chosen contact. From the QRSMS app, this can be achieved by going from the inbox screen, select "Generate QR", then select or input the contact that this keypair will be created for. This will then forward the user to a screen that shows a QR code that contains the public key from the generated keypair. They can show the QR code to the person, save it to their device or share it via other application (fig. 4).

After receiving their contact's QR Code, they can scan the given code through their mobile phone's camera (fig. 5). The scanning process will check if the code is a valid code, and then attempt to initiate an Elliptic Curve Diffie-Hellman (ECDH) key agreement with the said contact. Once this step is done for both users, they can now begin exchanging encrypted messages.

1) *Key pair generation:* In order to generate an EC key pair for a given contact, the `KeyPairGenerator` class from the `java.security` package was used. Before the final key

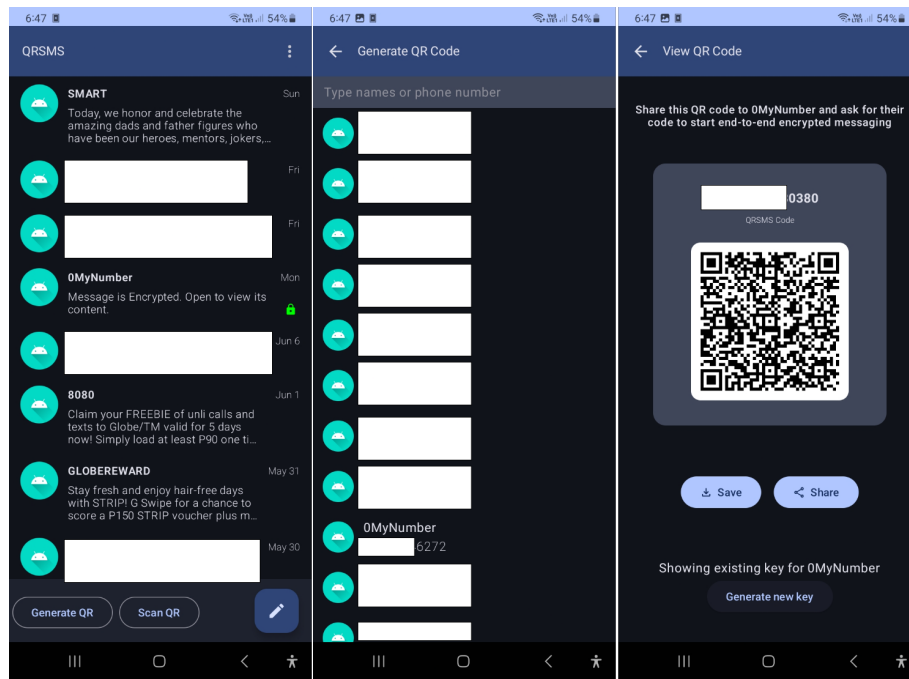


Fig. 4. QR Code Generation Process

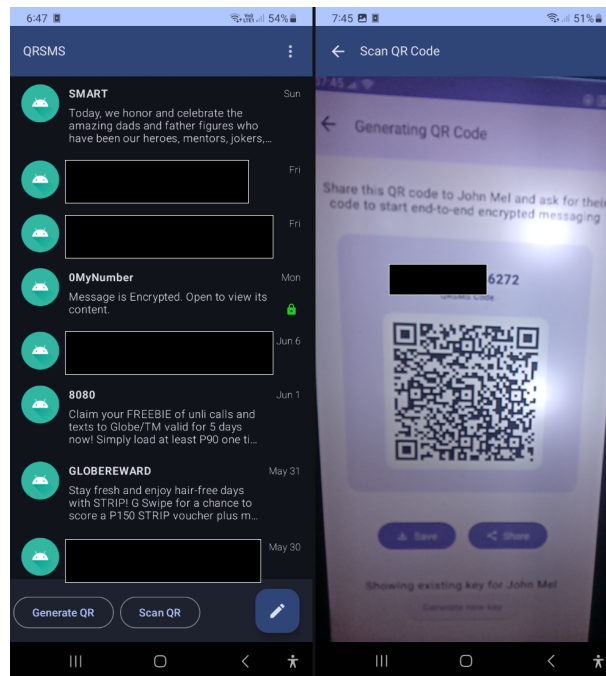


Fig. 5. QR Code Scanning Process

pair can be obtained, this class must first be instantiated and initialized. In the application, doing these actions can slightly differ depending on the device's API level.

API level 31 and above: If the device's API level is 31 or higher — that is, starting from android version 12, an instance of the `KeyPairGenerator` class can be instantiated with the `getInstance()` method by adding the algorithm name `EC` as the first parameter and giving the string `AndroidKeyStore` in

the provider field. The generator is then initialized by building an `AlgorithmParameterSpec` using the `KeyGenParameterSpec.Builder` class. An alias which is the identifier for the key when stored inside the `AndroidKeyStore` system is provided as well as the intended purpose of the key which is available as the constant `PURPOSE_AGREE_KEY` from the `KeyProperties` class are added to the builder. This ensures that the key material can only be used for the ECDH key-

agreement. The parameter spec for EC was set with the `setAlgorithmParameterSpec()` method so that the generated key pair will be under the `secp256r1` curve. Finally, the `generateKeyPair()` method of the generator class is invoked which creates a `KeyPair` entry inside the `AndroidKeyStore`.

API level 24 to 30: For devices with API level 24 to 30 (Android 7 to 11), the generation process involves the same instantiation and initialization. To get an instance of the generator, the `getInstance()` method is called with the standard algorithm name added as a parameter. In this case, adding a provider is optional. Similarly, when initializing the generator, only the `AlgorithmParameterSpec` is needed. A call can then be made to the `generateKeyPair()` method which returns a `KeyPair` object. However, because the `PURPOSE_AGREE_KEY` is only added from API 31 and above, the keypair is manually inserted to a Room database which is an abstraction layer for SQLite [34], and is encrypted using the `SQLCipher for android` [35] library.

2) *QR Code Generation:* Once a key pair is generated, the public key along with the phone number of the user is encoded in a QR code with the format of `<phone_number>:<public_key_in_base_64>`. Since the public key is a `PublicKey` object, it must first be encoded into byte array and then converted into a base64 since the qr code generator encodes a string.

QR Code generation was achieved using the `com.google.zxing` library adding it to the app as a dependency. With this library, the qr code generator can use the `qrcode.QRCodeWriter` class which contains the `encode()` method that can be used to generate the QR Code by passing the text to be encoded, format/type of the barcode, width and height, and additional hints that can be added to the QR Code. The encoded QR Code is converted into a `Bitmap` image so that it can be displayed to the user. This QR code can then be saved to the user's device or shared via other applications using android `Intents`.

3) *QR Code Scanning:* QR Code scanning capabilities are provided by android's `CameraX` with Google's `MLKit` analyzer and barcode scanning. The `CameraX` library enables the app to use the camera and attach its preview to a `PreviewView`. The `CameraX` package also provides built-in integration of Google's `MLKit` analyzer. By using the `setImageAnalysisAnalyzer()` method, the `BarcodeScannerOptions` is passed to the `MLKitAnalyzer` class so that the camera can detect QR Codes and decode its content. If the analyzer detects a valid QR Code, it will then attempt to perform a key exchange.

4) *Key-Agreement:* Upon decoding a valid QR code for QRSMS, the application attempts to perform an ECDH key agreement. Since the phone number of the contact is also embedded inside the QR code, the system checks if there is an existing key pair stored in either the room database or the android keystore system depending on the device's supported API level. If there is, the system performs ECDH key agreement by taking the user's private key for the contact and their public key as parameters. In the same step, the contact's public key gets parsed back into a byte array from

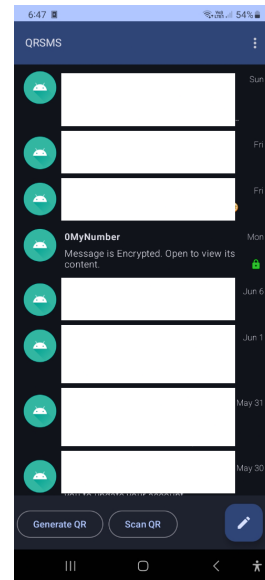


Fig. 6. QRSMS Inbox Screen

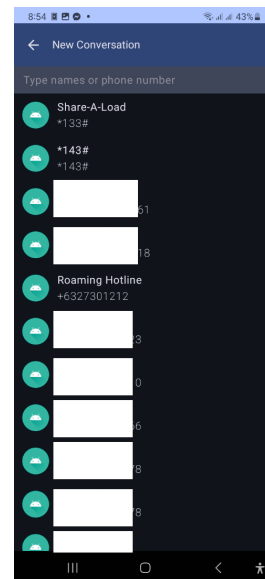


Fig. 7. QRSMS New Conversation Screen

the base 64 string that it is from the decoding step.

The `KeyAgreement` class contains the function needed to generate a shared secret key. To use it, an instance of this class was obtained by using the `getInstance()` method with the key-agreement algorithm name — "ECDH" as the parameter. This instance was initialized by giving the private key of the user as a parameter to the `init` method. Then, the `doPhase()` method is invoked with the contact's public key as the first parameter and a boolean value of true to signify that this is the last phase of the key-agreement. The `generateSecret()` method finally generates the shared secret key based on the key objects provided in the earlier phases. This secret key is further processed using through a HMAC-based Extract-and-Expand Key Derivation Function (HKDF) by adding the `at.favre.lib.hkdf` library. A

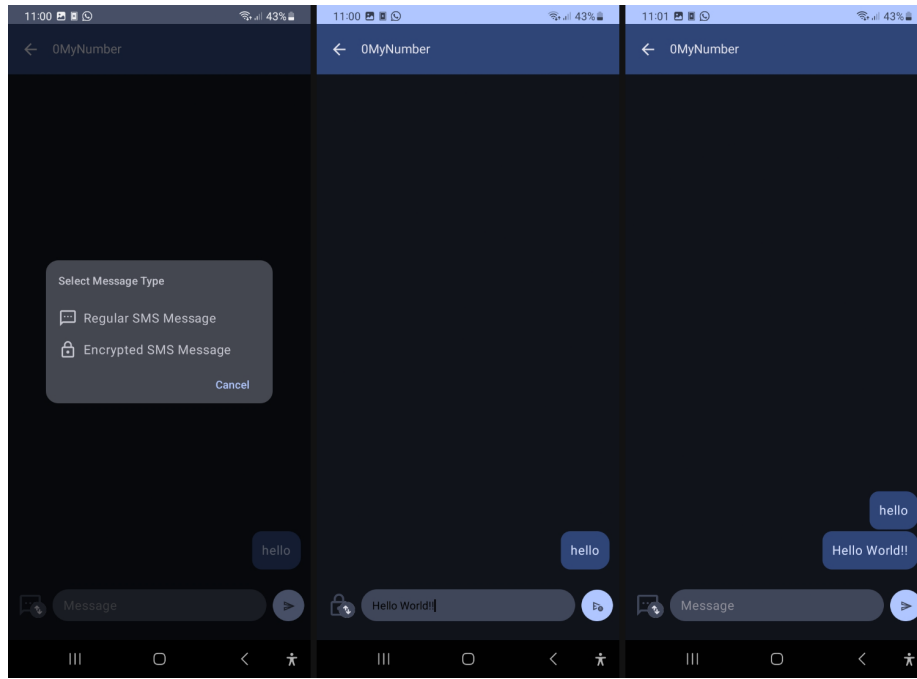


Fig. 8. Sending Encrypted Message

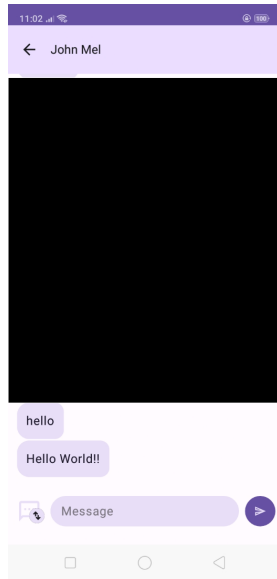


Fig. 9. Reading Encrypted Message

salt based on the phone numbers of the users involved in the exchange along with the generated shared secret key is given to the `hkdf extract` function to create a pseudorandom key. Before the expand phase, the algorithm string, the user and their contact's public key bytes are combined to form the info parameter. The pseudorandom key, info, and length of 32-bytes is provided as parameter to the `hkdf expand` function which outputs the final secret key material of the given length that will be used for the encryption and decryption algorithm. The derived key is then stored inside the `AndroidKeyStore` for encryption and decryption, using the contact's phone number as the key's alias.

B. SMS Client

While the generation and scanning of QR code is described in the previous sections, the client also provides users the ability to send and read regular and encrypted messages. Messages are retrieved from the default SMS application's content provider to ensure that the built-in SMS application remains operational even with QRSMS installed. To send a message, the user can either select a conversation from the inbox screen (fig. 6) which will take the user to the conversation screen that displays the thread of past messages, or select the create message icon in the lower right portion of the screen. Using the new conversation approach moves the user to a screen where they can select a contact or enter the phone number of the recipient (fig. 7).

In the conversation screen, if the user has an existing secret key with the contact (e.g. they have previously performed a key-agreement), a select message type icon appears in front of the message input box which will show a dialog where the user can select either to send a regular or an encrypted SMS. Regardless of what type of message the user chooses to send, sending the message is done through the `SmsManager` class that the android system provides. However, choosing to send an encrypted SMS message includes additional processing to the text before sending it to the recipient (fig. 8). On the other hand, decryption of SMS message is done automatically by the application such that when the user opens a conversation that includes an encrypted text, the application will automatically attempt to decrypt the message, given that the key used to encrypt the message is available (fig. 9). If the same message is attempted to be viewed from the default SMS app, or if the corresponding key is not available (e.g the key is deleted, the user has not scanned the code for that specific contact), users will only see a string of text in base 64 (fig. 10).

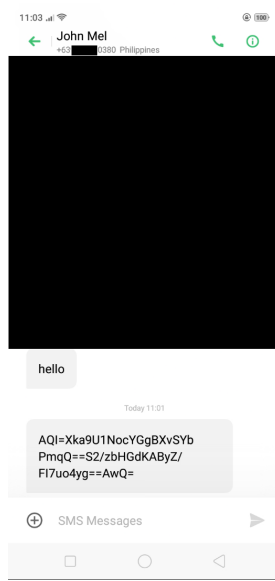


Fig. 10. Encrypted message as viewed in default sms app



Fig. 11. Key Management Screen

The app also provides a way to view and manage existing keys. The key management screen can be accessed by pressing the vertical ellipsis at the top right of the screen which brings out a menu where Manage Keys can be selected (fig. 11). Users can view the QR code which forwards them to the screen displayed after generating a QR Code and also a delete button. The screen also gives indication if end-to-end encryption is available for a given contact.

1) *Message Encryption*: If the user choose to send an encrypted message, the text is encrypted using the Cipher class from the javax.crypto package. Android supports the "AES/CBC/PKCS7Padding" transformation which is used to create an instance of the cipher class. It is then initialized using the init() method providing the ENCRYPT_MODE constant to indicate that the cipher object is used for encryption

as well as the secret key object obtained from Android KeyStore. The doFinal() method encrypts the message and returns the ciphertext in byte array. The cipher object also automatically generates an initialization vector (IV) for each encryption. The IV is prepended before the cipher text and converted into base 64 string. Before sending the encrypted message to the receiver, the author chose the Start of Heading + Start of Text unicode characters to indicate the beginning of the encrypted message and the End of Text + End of Transmission characters to indicate the end. This characters are also converted to base 64 so that the SMS message will not be converted to UTF-8 encoding which reduces the maximum length of a message from 160 to just 70.

2) *Message Decryption*: Decryption of a message is done automatically when a conversation is opened. When rendering a message to the screen, the application checks if the current message is encrypted. Encrypted messages are identified by checking if the given text has the prepended and appended characters. If it is, the flags are striped from the ciphertext. Before initializing the cipher object, the IV must be extracted from the base 64 string and then reconstructed using the IvParameterSpec. With the IV extracted, the cipher object can now be initialized with DECRYPT_MODE as its mode of operation, the secret key shared with the contact, and the extracted iv. The doFinal() method can then be performed on the cipher text which will decrypt the given message.

C. QRSMS Usability Testing

During the testing period, 14 respondents had tested and evaluated the application. Tables II and III shows the result of users' response for the SEQ and SUS respectively.

1) *Single Ease Question*: Users were given a total of 9 tasks as shown in table I where they are asked how they found its difficulty after the completion of each task. On average, users found each task with relative ease scoring an average of greater than 5 for most of the task except for tasks 6 and 7 with an average score of 4.43 and 4.50 respectively, which are related to sending SMS message. This could be due to a technical error in the application where in some cases, messages could not be sent. This assumption cannot be made certain though as there were no follow-up questions to ask users why they gave the task a certain score. Overall, users generally found the application fairly easy to use.

2) *System Usability Scale*: After testing out all core functionality of the system, users were asked to answer a standard System Usability Scale. SUS scores are calculated for each user where the scale value for odd numbered items were subtracted by 1, while the value for even numbered items were subtracted to 5. The difference is multiplied by 2.5 and is summed to get the overall usability of the system for a user [41]. In total, the average SUS score for QRSMS is 58.93. This suggests that users' overall experience with the system is not the worst but also not the best [42].

VI. CONCLUSION

This paper has shown a different approach to end-to-end encryption via the use of QR codes. The author was able to

No.	Task	Task Description
1	View conversation	Select a conversation to view in the inbox screen or create one by composing a new message.
2	Generate QR Code	Generate a QR code for a selected contact
3	Save generated QR Code	After the QR Code generation task, save the QR code to the device's gallery
4	Share generated QR Code	After the QR Code generation task, share the QR code through other platform using the share function
5	Scan QR Code	Scan a valid QRSMS code received from a contact using your camera
6	Send a regular SMS message	Select a conversation in the inbox screen or compose a new message to a contact
7	Send an encrypted SMS message	After giving a contact your QR code and scanning their's, switch message type to Encrypted Message, and send an encrypted SMS message to that contact
8	View generated QR Code	Find the "Manage Keys" screen and view the generated QR Code of a contact
9	Delete QR Code	Find the "Manage Keys" Screen and delete the generated QR Code of a contact

TABLE I
TASKS TO TEST

Repondent No.	Task								
	T1	T2	T3	T4	T5	T6	T7	T8	T9
1	5	7	7	5	4	4	3	5	7
2	4	4	4	4	4	1	1	4	4
3	7	7	7	7	7	7	7	6	6
4	4	4	4	4	4	4	4	4	4
5	1	1	1	1	1	1	1	1	1
6	7	6	6	7	6	7	7	6	6
7	7	7	7	7	7	7	7	7	7
8	6	7	7	7	7	5	5	5	7
9	7	7	7	6	6	7	7	6	7
10	1	1	1	1	1	1	1	1	1
11	4	3	7	7	6	3	5	7	7
12	7	7	7	7	7	5	5	7	7
13	7	7	5	6	5	3	5	7	5
14	7	6	7	6	7	7	5	7	7
Average	5.29	5.29	5.50	5.36	5.14	4.43	4.50	5.21	5.43

TABLE II
SUMMARY OF RESPONSE FOR SINGLE EASE QUESTION

Respondent No.	Question										SUS Score
	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	
1	3	2	4	4	2	4	5	4	5	3	55.00
2	3	4	4	5	3	4	2	3	3	4	37.50
3	4	2	4	2	4	2	4	3	4	1	75.00
4	5	5	5	5	5	5	5	5	5	5	50.00
5	3	3	3	3	3	3	3	3	3	3	50.00
6	4	4	4	3	4	3	4	2	4	3	62.50
7	4	1	5	2	4	2	4	2	3	1	80.00
8	4	2	5	4	4	2	4	3	4	1	72.50
9	4	3	4	3	5	2	5	2	3	3	70.00
10	3	3	3	3	3	3	3	3	3	3	50.00
11	3	2	3	4	3	2	3	3	4	3	55.00
12	3	2	4	3	4	3	2	2	4	2	62.50
13	3	4	2	4	2	4	2	4	2	2	32.50
14	4	3	3	2	4	2	3	1	5	2	72.50
Average SUS Score											58.93

TABLE III
SUMMARY OF RESPONSE FOR SYSTEM USABILITY SCALE AND OVERALL SUS SCORE

develop an android application that is capable of generate key pairs which can be encoded in a QR code, decode one from a contact to initiate an ECDH key agreement. The client is also capable of sending and reading regular and encrypted SMS based on the user's needs. Based on the result of the usability testing, it seems that despite being relatively easy to use for the average user scoring an average greated than 5 except for 2 tasks, not many may find it very appealing to use for their everyday use with a average SUS score of 58.93.

VII. RECOMMENDATIONS AND FUTURE WORKS

Users generally have different needs and requirements when it comes to the apps that they use for their daily lives. For future researches, it might be beneficial if the system is implemented and tested in an environment where confidentiality is much more critical than the average user. The idea of using SMS and QR codes to enable end-to-end encryption might also be of use in other domains such as Internet-of-things for example. This paper has shown how such system can be utilized for a more generalized use case such as person-to-person communication. However, more specialized fields might benefit more from the advantage that this system brings. Future works based on this system might also want to consider a different approach when testing the application.

The QRSMS client itself are also open for more feature set as well as some quality of life improvements. At its current state, features such as being able to switch between sim cards on demand, being able to add an image from the device's storage for QR scanning, and better and user-friendly prompts that requires user's attention can be added among many others. Overall, QRSMS is a product that shows a different approach to end-to-end security using existing tools and technology.

REFERENCES

- [1] G. Le Bodic, *Mobile messaging technologies and services: SMS, EMS, and MMS*, 2nd ed. Chichester, West Sussex, England; Hoboken, NJ: J. Wiley & Sons, 2005, ch. 3, p. 47.
- [2] S. Herring, D. Stein, and T. Virtanen, *Pragmatics of Computer-Mediated Communication*. Walter de Gruyter, Jan. 2013, ch. 7, p. 162, google-Books-ID: 9cTmBQAAQBAJ.
- [3] C. W. Yoo, J. Lee, C. Yoo, and N. Xiao, "Coping behaviors in short message service (sms)-based disaster alert systems: From the lens of protection motivation theory as elaboration likelihood," *Information & Management*, vol. 58, no. 4, p. 103454, June 2021.
- [4] S. Azid, B. Sharma, K. Raghuwaiya, A. Chand, S. Prasad, and A. Jacquier, "Sms based flood monitoring and early warning system," *ARPJ Journal of Engineering and Applied Sciences*, vol. 10, no. 15, 2015.
- [5] D. Gargaro, "What is sms for business?" Sept. 2023. [Online]. Available: <https://www.business.com/articles/sms-for-business/>
- [6] S. Okazaki and C. R. Taylor, "What is sms advertising and why do multinationals adopt it? answers from an empirical study in european markets," *Journal of Business Research*, vol. 61, no. 1, p. 4–12, Jan. 2008.
- [7] N. Saxena and N. S. Chaudhari, "Securesms: A secure sms protocol for vas and other applications," *Journal of Systems and Software*, vol. 90, p. 138–150, Apr. 2014.
- [8] J. Tuckerman, K. Harper, T. R. Sullivan, A. R. Cuthbert, J. Fereday, J. Couper, N. Smith, A. Tai, A. Kelly, R. Couper, M. Friswell, L. Flood, C. C. Blyth, M. Danchin, and H. S. Marshall, "Short message service reminder nudge for parents and influenza vaccination uptake in children and adolescents with special risk medical conditions: The flutext-4u randomized clinical trial," *JAMA Pediatrics*, vol. 177, no. 4, p. 337–344, Apr. 2023.
- [9] S. L. King, J. Lebert, L. A. Karpisek, A. Phillips, T. Neal, and K. Kosyluk, "Characterizing user experiences with an sms text messaging-based mhealth intervention: Mixed methods study," *JMIR Formative Research*, vol. 6, no. 5, p. e35699, May 2022.
- [10] A. Onashoga, A. Ogunjobi, T. Ibharalu, and O. Lawal, "A secure framework for sms-based service delivery in m-government using a multicast encryption scheme," *African Journal of Science, Technology, Innovation and Development*, vol. 8, no. 3, p. 247–255, June 2016.
- [11] T. D. Susanto and R. Goodwin, "User acceptance of sms-based e-government services: Differences between adopters and non-adopters," *Government Information Quarterly*, vol. 30, no. 4, p. 486–497, Oct. 2013.
- [12] J. B. Hill, C. M. Hill, and D. Sherman, "Text messaging in an academic library: Integrating sms into digital reference," *The Reference Librarian*, vol. 47, no. 1, p. 17–29, July 2007.
- [13] A. Rhoda Iyanda and M. Ebenezer Fasasic, "Development of two-factor authentication login system using dynamic password with sms verification," *International Journal of Education and Management Engineering*, vol. 12, no. 3, p. 13–21, June 2022.
- [14] "Empowering customers with everyday digital solutions," Globe Telecom, Inc., Apr. 2023. [Online]. Available: <https://www.globe.com.ph/content/dam/globe/brie/AboutUs/investor-relations/integrated-report/Globe-2022-Integrated-Report.pdf>
- [15] E. F. Rosales, "Text messaging remains relevant in philippines," Nov. 2022. [Online]. Available: <https://www.philstar.com/business/2022/11/25/2226194/text-messaging-remains-relevant-philippines>
- [16] *Short Message Service (SMS) for Wideband Spread Spectrum Systems*, 3GPP2 Technical Requirement C.S0015-C, Nov. 2012. [Online]. Available: https://www.arib.or.jp/english/html/overview/doc/STD-T64v6_20/Specification/ARIB_STD-T64-C.S0015-Cv1.0.pdf
- [17] J. Schiller, *Mobile communications*, 2nd ed. London: Addison-Wesley, 2011, pp. 120–122.
- [18] A. Biryukov, A. Shamir, and D. Wagner, "Real time cryptanalysis of a5/1 on a pc," in *Fast Software Encryption*, ser. Lecture Notes in Computer Science, G. Goos, J. Hartmanis, J. van Leeuwen, and B. Schneier, Eds. Berlin, Heidelberg: Springer, 2001, p. 1–18.
- [19] A. Bogdanov, T. Eisenbarth, and A. Rupp, "A hardware-assisted realtime attack on a5/2 without precomputations," in *Cryptographic Hardware and Embedded Systems - CHES 2007*, ser. Lecture Notes in Computer Science, P. Paillier and I. Verbauwhede, Eds. Berlin, Heidelberg: Springer, 2007, p. 394–412.
- [20] E. Barkan, E. Biham, and N. Keller, "Instant ciphertext-only cryptanalysis of gsm encrypted communication," in *Advances in Cryptology - CRYPTO 2003*, ser. Lecture Notes in Computer Science, D. Boneh, Ed. Berlin, Heidelberg: Springer, 2003, p. 600–616.
- [21] O. Dunkelman, N. Keller, and A. Shamir, "A practical-time attack on the a5/3 cryptosystem used in third generation gsm telephony," Cryptology ePrint Archive, Paper 2010/013, 2010, publication info: Published elsewhere. Unknown where it was published. [Online]. Available: <https://eprint.iacr.org/2010/013>
- [22] *Digital cellular telecommunications system (Phase 2+); Security mechanisms for SIM application toolkit; Stage 2*, European Telecommunications Standards Institute Technical Specification 03.48, Dec. 2001. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/101100_101199/101181/08.08.00.60/ts_101181v080800p.pdf
- [23] G. Cattaneo, G. De Maio, P. Faruolo, and U. F. Pterillo, "A review of security attacks on the gsm standard," in *Information and Communication Technology*, ser. Lecture Notes in Computer Science, K. Mustofa, E. J. Neuhold, A. M. Tjoa, E. Weippl, and I. You, Eds. Berlin, Heidelberg: Springer, 2013, p. 507–512.
- [24] H. Rongyu, Z. Guolei, C. Chaowen, X. Hui, Q. Xi, and Q. Zheng, "A pk-sim card based end-to-end security framework for sms," *Computer Standards & Interfaces*, vol. 31, pp. 629–641, June 2008.
- [25] J. L.-C. Lo, J. Bishop, and J. Eloff, "Smssec: An end-to-end protocol for secure sms," *Computers & Security*, vol. 27, no. 5–6, p. 154–167, Oct. 2008. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0167404808000151>
- [26] N. Saxena and N. S. Chaudhari, "Easysms: A protocol for end-to-end secure transmission of sms," *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 7, p. 1157–1168, July 2014.
- [27] P. Vijayakumar, S. M. Ganesh, L. J. Deborah, and B. S. Rawal, "A new smartsms protocol for secure sms communication in m-health environment," *Computers & Electrical Engineering*, vol. 65, p. 265–281, Jan. 2018.
- [28] S. Tiwari, "An introduction to qr code technology," in *2016 International Conference on Information Technology (ICIT)*, Dec.

- 2016, p. 39–44. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7966807>
- [29] What is a qr code. [Online]. Available: <https://www.qrcode.com/en/about/>
- [30] D.-H. Shin, J. Jung, and B.-H. Chang, “The psychology behind qr codes: User experience perspective,” *Computers in Human Behavior*, vol. 28, no. 4, p. 1417–1426, July 2012.
- [31] S. N. Karale, K. Pendke, and P. Dahiwal, “The survey of various techniques & algorithms for sms security,” in *2015 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS)*, Mar. 2015, p. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7192943>
- [32] M. Amara and A. Siad, “Elliptic curve cryptography and its applications,” in *International Workshop on Systems, Signal Processing and their Applications, WOSSPA*, May 2011, p. 247–250. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5931464>
- [33] Android keystore system. [Online]. Available: <https://developer.android.com/privacy-and-security/keystore>
- [34] Room. [Online]. Available: <https://developer.android.com/jetpack/androidx/releases/room>
- [35] “sqlcipher-android.” [Online]. Available: <https://github.com/sqlcipher/sqlcipher-android>
- [36] zxing. [Online]. Available: <https://github.com/zxing/zxing>
- [37] Ml kit analyzer. [Online]. Available: <https://developer.android.com/media/camera/camerax/mlkitanalyzer>
- [38] Scan barcodes with ml kit on android. [Online]. Available: <https://developers.google.com/ml-kit/vision/barcode-scanning/android>
- [39] M. Moskala and I. Wojda, *Android Development with Kotlin*. Packt Publishing Ltd, Aug. 2017, pp. 8–9, google-Books-ID: PJZGDwAAQBAJ.
- [40] (2023, July) Recommendations for android architecture. [Online]. Available: <https://developer.android.com/topic/architecture/recommendations>
- [41] J. Brooke, “Sus: A quick and dirty usability scale,” *Usability Eval. Ind.*, vol. 189, Nov. 1995.
- [42] A. Bangor, “Determining what individual sus scores mean,” vol. 4, no. 3, 2009.



John Mel R. Ramos is a BS Computer Science student in the University of the Philippines Los Baños. With interest in problem solving through programming, he wants to focus on a path towards software development. He is a member of the Alliance of Computer Science Students.